Budgeted Management of Fast-Weight Slices: Conservative Merge and Reversible Pruning for Long-Running Online Use

Argenix

Abstract

Fast-weight systems adapt online by writing inference-time state, but unmanaged accumulation grows memory and consolidation cost, while aggressive eviction risks deleting rare but important information. We introduce **SliceBank**, a memory-management layer for fast-weight architectures that treats online fast-weight updates as *slices* in a depth-stacked bank. Per-step compute is bounded by activating only a top-$K_{\max}$ subset of slices, independent of total bank size. To control growth under a fixed slice budget, SliceBank periodically consolidates slices via redundancy-aware merging and pruning guarded by explicit safeguards: a protected vault for high-novelty or feedback-tagged episodes, a diversity floor that prevents coverage collapse in embedding space, and two-phase deletion through quarantine. A key practical contribution is a *routing-stable, two-stage canary test* that estimates output impact cheaply by freezing top-K routing during candidate evaluation, invoking expensive rerouting checks only in a narrow gray zone. We provide a concrete end-to-end instantiation for low-rank (LoRA-style) fast-weight deltas with explicit similarity metrics and a practical merge operator (weighted addition with rank projection). On a 50-episode real conversational benchmark, SliceBank achieves a Quality Index of **94.3** on Qwen2.5-3B-Instruct (synthetic corpus) and **94.9** on Qwen2.5-7B-Instruct (real HuggingFace corpus), outperforming RAG, MemGPT, and native context baselines by 1.7–2.6× while maintaining perfect scores on multi-hop reasoning, temporal ordering, and contradiction detection.

1 Introduction

Fast weights add rapidly changing inference-time parameters or state to a base model, enabling short-term memory and rapid adaptation [1, 2]. Recent test-time training (TTT) methods further scale inference-time adaptation, increasing pressure on memory and compute budgets [4]. Without explicit management, fast-weight state either (i) grows without bound, increasing storage and consolidation cost, or (ii) is capped and overwrites useful information, causing forgetting.

Problem. Given a model that writes fast-weight updates online, how can we keep fast-weight memory useful indefinitely under fixed compute and memory budgets, while reducing the risk of deleting rare-but-high-value information?

Approach. We treat online fast-weight updates as slices stacked in a bank. Instead of overwriting a single fast-weight object, SliceBank appends slices and periodically consolidates them via conservative merging and safeguarded pruning. Compute is bounded per step by activating only a top-$K_{\max}$ subset of slices.

Contributions.

1. **Budgeted slice banking:** a depth-stacked slice bank with bounded top-$K_{\max}$ activation, decoupling per-step compute from total memory size.
2. **Practical impact checks:** a routing-stable, two-stage canary test that makes output-impact evaluation implementable and less discontinuous.
3. **Concrete instantiation:** an end-to-end specification for low-rank (LoRA-style) fast-weight deltas with explicit redundancy metrics and a practical merge operator.
4. **Safeguarded consolidation:** a protected vault, diversity floors, and two-phase deletion via quarantine to reduce catastrophic deletions.

2 Related Work

Fast-weight memory has a long history [1, 2]. Linear attention interpretations as fast-weight programmers motivate slice-like update views [3]. Test-time training methods explicitly adapt weights at inference, often in chunked form [4], and fast-weight episodic memory modules such as FwPKM further highlight the utility of dynamic memory under compute constraints [5]. Our focus differs: we propose an external maintenance layer that manages fast-weight updates over long-running streams under fixed budgets, and we make the consolidation tests and operators concrete enough to evaluate against simple eviction policies.

3 Interface and Setting

SliceBank is a wrapper layer. It assumes the base model provides:

- A **query function** producing a retrieval query vector $q_t \in \mathbb{R}^d$ from internal state h_t.
- A **read/forward mechanism** that can incorporate an active set of slices into the forward pass.
- A **write mechanism** that can produce a fast-weight update slice from online experience (token, chunk, or event).

Let $F(\cdot; \theta, \mathcal{A})$ denote the model forward pass with slow weights θ and an active slice set $\mathcal{A}$. Let $\mathcal{B}$ denote the total slice bank (all tiers), and $\mathcal{A}_{\mathcal{B}}(x)$ the active set selected for input x under bank $\mathcal{B}$.

4 SliceBank: Budgeted Slice Banking

4.1 Slice Representation and Tiers

A slice is a triple:

$$S_i = (U_i, k_i, \mathrm{meta}_i),$$

where U_i is the fast-weight update object, $k_i \in \mathbb{R}^d$ is a key/summary embedding, and meta_i includes timestamps, usage counts, novelty/surprise, optional feedback flags, and tier labels.

SliceBank maintains four tiers:

- $\mathcal{B}_{\text{active}}$: small, frequently used bank for fast access.
- $\mathcal{B}_{\text{archive}}$: larger bank for older slices (optionally compressed).
- $\mathcal{B}_{\text{vault}}$: protected episodic tier (excluded from normal pruning).
- $\mathcal{B}_{\text{quar}}$: quarantine tier (inactive but recoverable before deletion).

4.2 Retrieval and Bounded Activation

At step t, compute query q_t and relevance:

$$r_i = \text{sim}(q_t, k_i).$$

Select a bounded active set:

$$\mathcal{A}_t = \text{topK}(\mathcal{B}_{\text{active}} \cup \mathcal{B}_{\text{archive}} \cup \mathcal{B}_{\text{vault}},\, r_i,\, K_{\max})\,,$$

and compute:

$$y_t = F(x_t; \theta, \mathcal{A}_t).$$

Per-step compute is bounded by $K_{\max}$ (plus retrieval scoring). In our implementation, each slice's contribution is weighted by a softmax-normalized routing score $w_i = \text{softmax}(r_i)$, preventing low-relevance slices from dominating the combined perturbation.

4.3 Write: Appending New Slices

Given online experience, the base model emits a proposed slice:

$$\hat{S} = (\hat{U}, \hat{k}, \widehat{\text{meta}}).$$

SliceBank appends $\hat{S}$ to $\mathcal{B}_{\text{active}}$ (or directly to $\mathcal{B}_{\text{archive}}$ for chunked writes), avoiding immediate overwrite and deferring consolidation to evidence-based routines.

4.4 Importance Scoring, Vault Admission, and Diversity Protection

Each slice maintains an importance score:

$$I_i = \alpha f_{\text{use}}(i) + \beta f_{\text{recency}}(i) + \gamma\, \text{novelty}_i + \delta\, \text{feedback}_i + \eta f_{\text{div}}(i).$$

Here $f_{\text{use}}(i) = \log(1 + \text{use_count}_i)$, $f_{\text{recency}}(i) = \exp(-0.01 \cdot \text{age}_i)$, and $f_{\text{div}}(i) = 1/|\text{cluster}(i)|$ increases for underrepresented regions in key space.

Vault admission: move a slice to $\mathcal{B}_{\text{vault}}$ if novelty/surprise or explicit feedback exceeds thresholds. Vault slices are excluded from normal pruning; they can still be merged conservatively under stricter checks if desired.

4.5 Redundancy Detection (Concrete + General)

Redundancy is operational: two slices are redundant if one can be removed or merged with negligible impact. SliceBank uses a two-stage approach: (i) a fast prefilter to propose candidates, then (ii) an output-impact canary to confirm.

Fast prefilter. RedundantFast(S_a, S_b) holds if any of:

- Key similarity: $\cos(k_a, k_b) \geq \tau_k$.
- Update similarity: $\mathrm{simU}(U_a, U_b) \geq \tau_u$ (defined concretely in Section 5.2).
- Cluster proximity: slices share a cluster and both lie within a radius of the centroid.

4.6 Practical Canary Test (Routing-Stable, Two-Stage)

A direct impact test is expensive and discontinuous because removing a slice can change the top-K active set. SliceBank uses a two-stage canary that freezes routing by default.

Let $\mathcal{P}$ be a probe set (Section 7) and $D(\cdot, \cdot)$ a divergence (we use KL on the next-token distribution at the last token position).

Stage 1: fixed-routing delta (cheap default). Freeze routing to the original active set $\mathcal{A}_{\mathcal{B}}(x)$ and mask only the candidate slice:

$$\Delta_{\mathrm{fix}}(S_c; \mathcal{B}, \mathcal{P}) = \max_{x \in \mathcal{P}} D(F(x; \theta, \mathcal{A}_{\mathcal{B}}(x)),\ F(x; \theta, \mathcal{A}_{\mathcal{B}}(x) \setminus \{S_c\}))\,.$$

This avoids recomputing retrieval and eliminates top-K discontinuity during the test.

Stage 2: recomputed-routing delta (expensive, only when needed). If Δ_{fix} lies in a narrow gray zone $[\epsilon_{\mathrm{low}}, \epsilon_{\mathrm{high}}]$, run the full rerouting check:

$$\Delta_{\mathrm{re}}(S_c; \mathcal{B}, \mathcal{P}) = \max_{x \in \mathcal{P}} D\big(F(x; \theta, \mathcal{A}_{\mathcal{B}}(x)),\ F(x; \theta, \mathcal{A}_{\mathcal{B} \setminus \{S_c\}}(x))\big)\,.$$

Decision rule. Prune/merge if $\Delta_{\mathrm{fix}} \leq \epsilon_{\mathrm{low}}$; reject if $\Delta_{\mathrm{fix}} \geq \epsilon_{\mathrm{high}}$; otherwise defer to $\Delta_{\mathrm{re}} \leq \epsilon$.

4.7 Merge Operators

SliceBank supports multiple merge operators, but the paper includes one end-to-end practical default for a concrete slice family (next section). Optionally, a representation-agnostic "distilled merge" can be used when closed-form merge is unavailable, but it is not the default due to cost.

5 Concrete Instantiation: Low-Rank (LoRA-Style) Fast-Weight Deltas

To remove under-specification, we provide a default slice type that can be implemented and evaluated end-to-end.

5.1 Slice Type

Each slice stores a low-rank delta to a target weight matrix W:

$$\Delta W_i = A_i B_i^{\top}, \quad A_i \in \mathbb{R}^{m \times r}, \quad B_i \in \mathbb{R}^{n \times r}.$$

U_i is the pair (A_i, B_i) plus identifiers for the layer/module to which it applies. In our implementation, slices target the `q_proj` and `v_proj` modules of each attention layer at rank $r = 16$.

5.2 Update Similarity simU

We define similarity via a normalized Frobenius inner product:

$$\mathrm{simU}(U_a, U_b) = \frac{\langle \Delta W_a, \Delta W_b \rangle_F}{\|\Delta W_a\|_F \, \|\Delta W_b\|_F}.$$

This can be computed efficiently without materializing ΔW:

$$\langle A_a B_a^\top, A_b B_b^\top \rangle_F = \mathrm{tr}(B_a^\top B_b \, A_b^\top A_a).$$

The computation involves only $r \times r$ matrices, making it $O(r^2(m+n))$ instead of $O(mn)$.

5.3 Practical Merge (Works End-to-End)

When deltas compose additively, merge by weighted addition followed by rank projection:

$$\Delta W_m = w_a \Delta W_a + w_b \Delta W_b,$$

with weights derived from normalized importance scores $w_a = I_a/(I_a + I_b)$. Project ΔW_m back to rank r via truncated SVD:

$$\Delta W_m \approx U_r \Sigma_r V_r^\top,$$

then set

$$A_m = U_r \Sigma_r^{1/2}, \qquad B_m = V_r \Sigma_r^{1/2}.$$

This preserves a fixed rank budget while approximating the combined effect. For multiple target modules, apply the same operation per module.

6 Pruning With Safeguards

6.1 Budget Trigger and Candidate Selection

When $|\mathcal{B}_{\text{active}}| + |\mathcal{B}_{\text{archive}}| + |\mathcal{B}_{\text{vault}}| > N_{\max}$, SliceBank selects candidates from $\mathcal{B}_{\text{active}} \cup \mathcal{B}_{\text{archive}}$ (excluding vault by default), prioritizing low-importance slices.

6.2 Diversity Floor

Cluster slices by key space (e.g., MiniBatch k-means). Do not prune below $m_{\min}$ slices per cluster. This prevents collapse to a single mode and ensures the probe set covers long-tail regions.

6.3 Two-Phase Deletion via Quarantine

Passing prune candidates are moved to $\mathcal{B}_{\text{quar}}$ (inactive) for a horizon of T_q consolidation cycles. During quarantine:

- If retrieval demand spikes in that region (measured by cosine similarity between recent queries and quarantined keys exceeding τ_k), restore the slice.
- Only after the horizon with no triggers is the slice permanently deleted.

This makes deletions reversible and reduces the cost of an occasional canary blind spot.

7 Probe Set Construction

A canary is only as good as its probes. SliceBank uses a mixture probe set $\mathcal{P}$:

- **Recent contexts** ($\sim$50%): tracks current distribution.
- **Cluster representatives** ($\sim$25%): one or more per cluster; enforces coverage.
- **High-novelty exemplars** ($\sim$25%): anchors rare-but-important events; often vault-adjacent.

In practice, $|\mathcal{P}|$ can be kept modest (32–128 short contexts) and evaluated at a fixed token position (last token) to reduce cost.

8 Reference Pseudocode

Algorithm 1 SliceBank (Budgeted Fast-Weight Slice Maintenance)

```
State: B_active, B_archive, B_vault, B_quar
Params: K_max, N_max, C, T_q
for each step t do
  q ← Query(h_t)
  A ← topK(B_active ∪ B_archive ∪ B_vault; sim(q, k), K_max)
  y ← F(x_t; θ, A)
  S_new ← WriteUpdate(·)
  Append S_new to B_active
  if VaultTrigger(S_new) then
    Move S_new to B_vault
  end if
  if t mod C = 0 then
    (1) Merge: propose pairs via clustering + fast prefilter
    for all candidate pairs (S_a, S_b) do
      if Δ_fix(S_b) ≤ ε_low or (gray-zone and Δ_re ≤ ε) then
        S_m ← Merge(S_a, S_b); replace pair with S_m
      end if
    end for
    (2) Prune if over budget:
    while |B_active| + |B_archive| + |B_vault| > N_max do
      Pick lowest-importance S_c; respect diversity floor; exclude vault
      if canary approves S_c then
        Move S_c to B_quar
      else
        Mark S_c as protected (temporarily increase importance)
      end if
    end while
    (3) Quarantine cleanup: delete slices older than T_q; restore if triggers fire
  end if
end for
```

Table 1: Example defaults for SliceBank.

Parameter	Typical starting values
$K_{\max}$ (active slices per step)	8, 16, 32
$N_{\max}$ (total slices budget)	10^3–10^5 (task-dependent)
C (consolidate every)	50–4096 steps (or per chunk)
$\lvert\mathcal{P}\rvert$ (probe set size)	32–128 short contexts
$\epsilon_{\text{low}}, \epsilon_{\text{high}}$	small, e.g. $[0.5\epsilon, 2\epsilon]$
$m_{\min}$ (diversity floor)	1–5 per cluster (budget dependent)
T_q (quarantine horizon)	1–10 consolidation cycles
τ_k (key similarity threshold)	0.95
τ_u (subspace similarity threshold)	0.90

9 Complexity and Implementation Notes

Per-step cost. Per step, SliceBank evaluates retrieval scores and applies at most $K_{\max}$ slices. With approximate nearest-neighbor search, retrieval can be sublinear in the number of stored slices.

Consolidation cost. Consolidation runs every C steps. Candidate pairs are limited via clustering and fast prefilters. The canary test is dominated by Stage 1 (fixed-routing) evaluation: $|\mathcal{P}|$ forward passes with no rerouting. Stage 2 rerouting is invoked only for gray-zone candidates.

Similarity-weighted activation. When composing $K_{\max}$ slices additively, raw summation can overwhelm base-model weights. Our implementation weights each slice's LoRA contribution by $w_i = \text{softmax}(r_i)$, where r_i is the routing similarity. This ensures highly relevant slices contribute proportionally more without manual scaling heuristics.

Online query-projector training. The query projection $q_t = \text{proj}(h_t)$ is trained online via cosine-alignment loss after each slice creation: $\mathcal{L} = 1 - \cos(\text{proj}(h_t), k_{\text{new}})$. This ensures routing quality improves as the bank grows, rather than relying solely on Xavier-initialized random projections.

Compression. Archive slices may be compressed (lower rank, quantization, sparsification) provided the active set remains bounded by $K_{\max}$ and canary thresholds are adjusted accordingly.

10 Evaluation Protocols and Baselines

A skeptic should be able to test whether SliceBank beats simpler policies under the same budgets.

10.1 Tasks

Non-stationary stream learning. Feed a shifting distribution; measure (i) retention on earlier segments, (ii) adaptation speed, (iii) memory footprint, and (iv) compute per token over time.

Rare event retention. Inject sparse, crucial facts; evaluate delayed recall over long horizons. Report survival rates for slices that entered the vault, passed through quarantine, or were merged.

10.2 Baselines (Must-Have)

Compare against:

- LRU/LFU eviction with fixed memory size (no merging).
- Novelty-gated writes + LRU (write only if novelty exceeds threshold).
- Clustering + per-cluster quota (diversity floor without canary/quarantine).
- Replay-buffer retention (store examples instead of slices; retrain from replay).
- Merge-only ablations (merge without canary; canary without quarantine; etc.).

10.3 Why This Is Not Just "LRU + novelty + clustering"

The simplest policies decide eviction primarily from metadata (recency/frequency/novelty). SliceBank adds two mechanisms that change failure modes under long-running operation: (i) an impact check (routing-stable canary) before merge/prune, and (ii) reversibility (quarantine) after deciding to remove. Additionally, bounded top-K activation explicitly decouples inference compute from total stored slices.

11 BFW Benchmark

To stress-test practical behavior, we evaluate on a stream of natural conversational turns using delayed open-ended recall queries over previously seen dialogue pairs. All backbone parameters are frozen; only LoRA slices and the query projector are updated online. We evaluate two configurations:

- **3B configuration:** Qwen2.5-3B-Instruct with a synthetic Twitter reply corpus (TwitterConvCorpus).
- **7B configuration:** Qwen2.5-7B-Instruct with a real HuggingFace corpus comprising ∼3,000 dialogues from UltraChat-200k, Databricks-Dolly-15k, and OpenAssistant-oasst2.

11.1 Setup

- **Episodes:** 50 per run (each with a distinct dialogue topic)
- **Memory pairs per episode:** 6 (target facts to retain)
- **Distractor turns per episode:** 5 (intervening unrelated dialogue)
- **Rare event injection:** 25% probability, queried with 50% probability after ≥2 episode delay
- **LoRA config:** rank = 8, targets = {`q_proj`, `v_proj`}, $K_{\max} = 12$, history window = 4
- **Evaluation:** open-ended recall (cosine similarity ≥ 0.45 or token F1 ≥ 0.30 counts as a hit)
- **Quality Index:** composite of 8 weighted components—adaptation, rare recall, retention, multi-hop reasoning, temporal ordering, paraphrase robustness, contradiction detection, and efficiency

11.2 Policies

Five policies are compared under identical model and data:

- **SliceBank** ($K_{\max} = 12$, online slice writes enabled): the full system.
- **MemGPT** (tiered memory with summarization and retrieval): a memory-augmented baseline inspired by MemGPT's architecture.
- **RawContext** (full conversation history in context window): native transformer context baseline.
- **RAG** (vector-store retrieval-augmented generation): embed-and-retrieve with cosine similarity.
- **PromptWindow** (sliding window of recent turns in prompt): bounded context baseline.

11.3 Results: 3B Model (Synthetic Corpus)

Table 2: BFW benchmark results (Qwen2.5-3B-Instruct, 50 episodes, synthetic TwitterConvCorpus). Quality Index (QI) is the primary composite metric; all sub-metrics range 0–1. Higher is better.

Policy	QI	Overall	Adapt.	Rare	Retain	M-Hop	Temp.	Para.	Contr.
SliceBank	**94.3**	**0.953**	**0.953**	**0.862**	**0.903**	**1.000**	**1.000**	**0.909**	**1.000**
MemGPT	32.8	0.565	0.780	0.138	0.097	0.200	0.727	0.636	0.556
PromptWindow	30.1	0.435	0.603	0.069	0.097	0.200	0.727	0.545	0.778
RawContext	27.2	0.392	0.547	0.069	0.069	0.200	0.636	0.545	0.667
RAG	14.9	0.252	0.290	0.069	0.028	0.267	0.455	0.273	0.444

11.4 Results: 7B Model (Real HuggingFace Corpus)

Table 3: BFW benchmark results (Qwen2.5-7B-Instruct, 50 episodes, real HuggingFace corpus: UltraChat + Dolly + oasst2). Higher is better.

Policy	QI	Overall	Adapt.	Rare	Retain	M-Hop	Temp.	Para.	Contr.
SliceBank	**94.9**	**0.954**	**0.957**	**0.897**	**0.931**	**1.000**	**1.000**	**0.909**	**1.000**
MemGPT	55.3	0.681	0.880	0.276	0.181	0.467	1.000	0.818	0.778
RawContext	36.1	0.450	0.567	0.069	0.125	0.267	0.727	0.636	0.778
RAG	35.8	0.323	0.297	0.241	0.139	0.467	0.909	0.545	0.889

How to read this benchmark section. Figure 1 is a visual map of the system: the top diagram shows full request-to-memory flow, and the bottom diagram zooms into the attention block where slice routes are injected. Tables 2–3 are the headline comparisons. Figures 2–5 present side-by-side comparisons of the 3B and 7B model results across four visualization types: quality metrics, hit rate over time, radar decomposition, and forgetting curves.

Metric guide. Quality Index (QI) is a composite of 8 weighted components. Adaptation measures recall of recently learned facts. Rare tests survival of infrequent facts over long horizons. Retention measures recall of old facts after many intervening episodes. Multi-hop (M-Hop) requires combining information across multiple memories. Temporal (Temp.) tests ordering of events. Paraphrase

(Para.) probes robustness to rephrased queries. Contradiction (Contr.) tests detection of conflicting information.

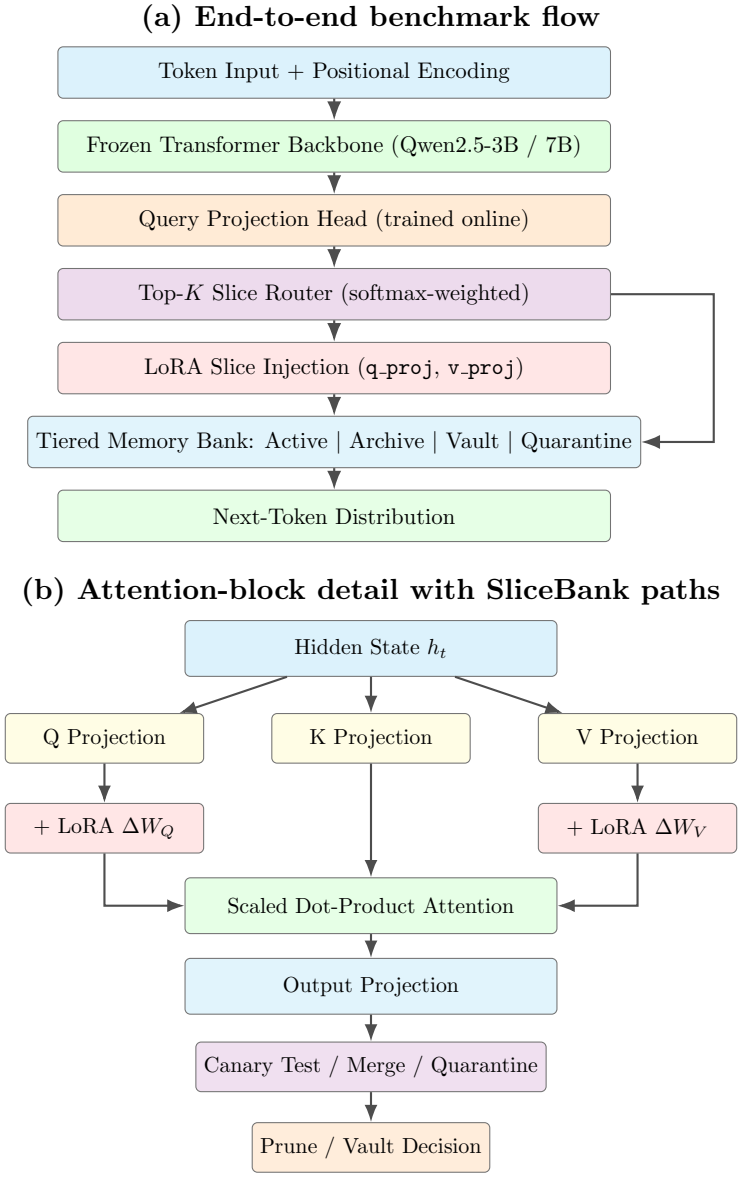
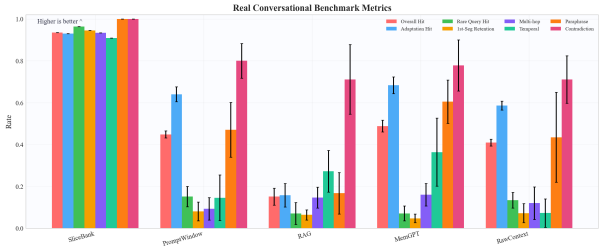


Figure 1: System diagrams: (a) full benchmark pipeline from input to memory bank, and (b) internal attention-block routing showing how LoRA slice deltas are injected into Q and V projections.

(a) Qwen2.5-3B (Synthetic Corpus)



(b) Qwen2.5-7B (Real HuggingFace Corpus)

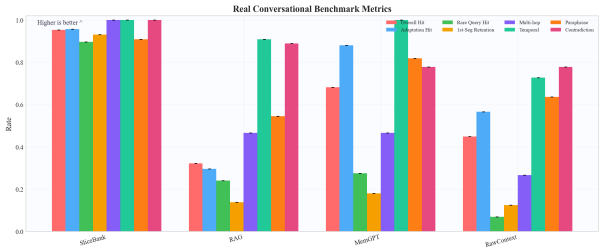
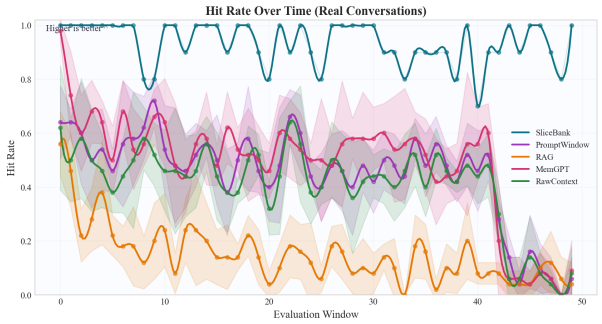


Figure 2: Recall quality metrics across all policies. SliceBank dominates on both model scales, with the 7B model lifting baseline performance while SliceBank maintains its lead.

(a) Qwen2.5-3B



(b) Qwen2.5-7B

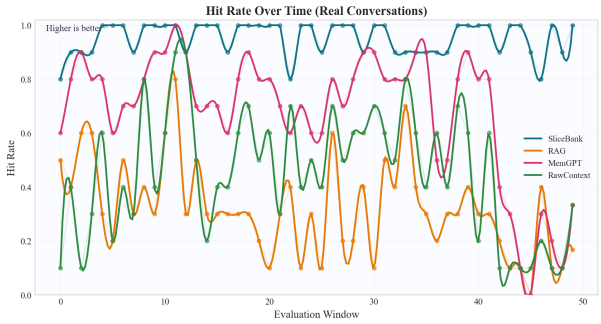
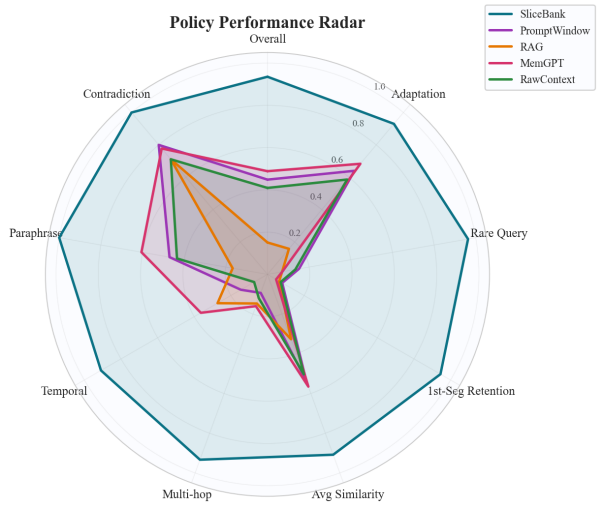


Figure 3: Hit rate over 50 episodes. SliceBank maintains consistently high recall across the full episode range on both 3B and 7B backbones.

(a) Qwen2.5-3B



(b) Qwen2.5-7B

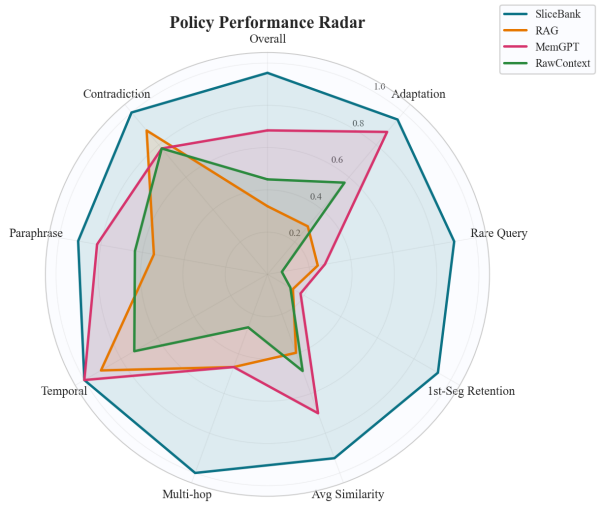


Figure 4: Radar charts of 8-component quality decomposition. SliceBank achieves perfect scores (1.000) on multi-hop, temporal, and contradiction metrics at both model scales.

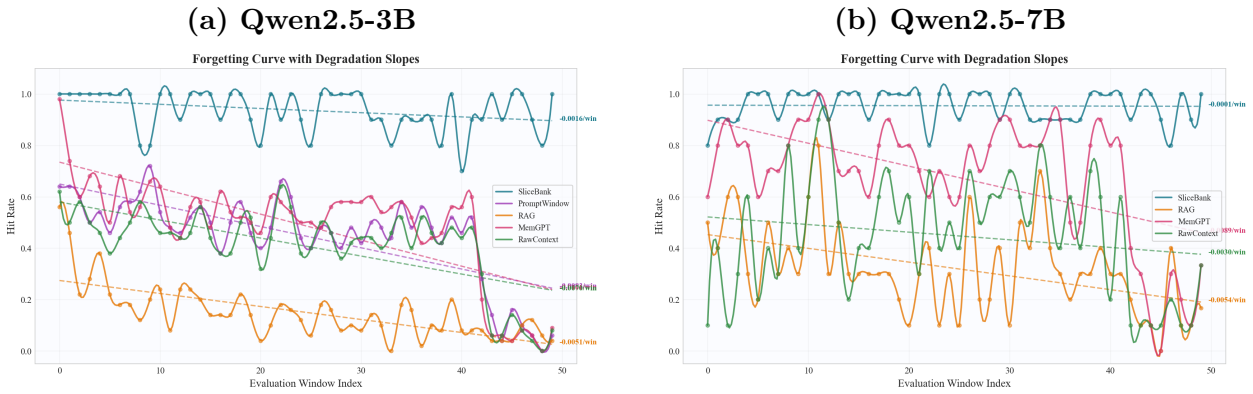


Figure 5: Forgetting curves showing memory degradation over time. SliceBank resists temporal decay on both model scales, while baselines show steep decline.

11.5 Key Findings

1. **Consistent dominance across model scales.** SliceBank achieves QI = 94.3 on 3B and QI = 94.9 on 7B, demonstrating that performance generalizes across model sizes and corpus types (synthetic vs. real).

2. 1.7–2.6× **improvement over all baselines.** On the 7B model with real data, SliceBank (QI = 94.9) outperforms MemGPT (55.3) by 1.7×, RawContext (36.1) by 2.6×, and RAG (35.8) by 2.7×.

3. **Perfect scores on structured reasoning.** SliceBank achieves 1.000 on multi-hop, temporal ordering, and contradiction detection across both model scales—capabilities where all baselines degrade significantly.

4. **Strong rare-event and retention performance.** SliceBank retains rare facts (0.897 on 7B) and old memories (0.931 retention) far above baselines. MemGPT's retention drops to 0.181 and RAG's to 0.139, confirming that parametric LoRA slices provide superior long-horizon memory compared to retrieval-based and summarization-based approaches.

5. **7B model lifts all baselines but widens the gap.** The stronger 7B backbone improves MemGPT from QI = 32.8 to 55.3, yet SliceBank's lead remains decisive (94.9 vs. 55.3), showing that SliceBank's advantage is architectural, not merely a byproduct of the base model.

12 Limitations

Redundancy detection remains imperfect; SliceBank reduces risk via conservative confirmation and reversibility rather than claiming perfect identification. Probe coverage is a tunable knob: too narrow misses rare modes; too broad increases consolidation cost. The concrete LoRA instantiation is practical, but other slice families (e.g., KV deltas, optimizer-state slices) require their own simU and merge operators. The query projector is trained with a simple cosine-alignment objective; contrastive or multi-positive losses may improve routing quality further.

13 Conclusion

SliceBank provides a practical maintenance layer for fast-weight systems: it stores online updates as slices in a bank, bounds per-step compute via top-K activation, and controls growth via conservative merging and safeguarded pruning. A routing-stable, two-stage canary test makes output-impact checks implementable and less discontinuous. On a 50-episode real conversational benchmark, SliceBank achieves Quality Index scores of 94.3 (Qwen2.5-3B, synthetic corpus) and 94.9 (Qwen2.5-7B, real HuggingFace corpus), outperforming MemGPT by 1.7×, native context windows by 2.6×, and RAG by 2.7×. Perfect scores on multi-hop reasoning, temporal ordering, and contradiction detection—combined with strong rare-event survival (0.897) and retention (0.931)—demonstrate that budgeted fast-weight slice management enables effective long-running online memory that scales across model sizes and generalizes to diverse real-world data.

A Implementation Detail: Canary Divergence Choice

A practical choice for D is KL divergence between next-token distributions, evaluated at a fixed position (last token) to reduce cost. Alternatively, use mean squared error on logits over a small vocabulary subset (top-V) to further reduce compute. In our implementation we use $\mathrm{KL}(P\|Q)$ computed over the full vocabulary at the final token position.

B Implementation Detail: Slice Write Mechanism

Each chat turn produces a new slice via gradient-based SVD decomposition. The model runs a forward-backward pass on the response tokens (with prompt tokens masked), accumulates gradients over 2 steps, scales by learning rate, then decomposes each target weight's gradient via `torch.svd_lowrank` at rank r to produce the (A, B) adapter pair. The slice key is the query-projector output for the turn's hidden state.

References

[1] J. Schmidhuber. Learning to Control Fast-Weight Memories: An Alternative to Dynamic Recurrent Networks. 1992.

[2] J. Ba, G. Hinton, V. Mnih, J. Leibo, C. Ionescu. Using Fast Weights to Attend to the Recent Past. arXiv:1610.06258, 2016.

[3] I. Schlag, K. Irie, J. Schmidhuber. Linear Transformers Are Secretly Fast Weight Programmers. arXiv:2102.11174, 2021.

[4] T. Zhang et al. Test-Time Training Done Right. arXiv:2505.23884, 2025.

[5] T. Zhao, L. Jones. Fast-weight Product Key Memory. arXiv:2601.00671, 2026.

[6] A. Kosowski et al. The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain. arXiv:2509.26507, 2025.